CHAPTER

10 Queues, memory management, and the ABA problem

10.1 Introduction

In the next few chapters, we look at a broad class of objects known as *pools*. A pool is similar to the `Set<>` class studied in Chapter 9, with two main differences: A pool does not necessarily provide a `contains()` method to test membership, and it allows the same item to appear more than once. The `Pool<>` has `put()` and `get()` methods, as shown in Fig. 10.1. Pools show up in many places in concurrent systems. For example, in many applications, one or more *producer* threads produce items to be consumed by one or more *consumer* threads. These items may be jobs to perform, keystrokes to interpret, purchase orders to execute, or packets to decode. Sometimes producers are *bursty*, suddenly and briefly producing items faster than consumers can consume them. To allow consumers to keep up, we can place a *buffer* between the producers and the consumers. Items produced faster than they can be consumed accumulate in the buffer, from which they are consumed as quickly as possible. Often, pools act as producer–consumer buffers.

Pools come in several varieties.

- A pool may be *bounded* or *unbounded*. A bounded pool holds a limited number of items. This limit is called its *capacity*. By contrast, an unbounded pool can hold any number of items. Bounded pools are useful when we want to keep producer and consumer threads loosely synchronized, ensuring that producers do not get too far ahead of consumers. Bounded pools may also be simpler to implement than unbounded pools. On the other hand, unbounded pools are useful when it is not easy to fix a limit on how far producers can outstrip consumers.
- Pool methods may be *total* or *partial*. Some partial methods are *synchronous*.

```
1 public interface Pool<T> {
2   void put(T item);
3   T get();
4 }
```

FIGURE 10.1

The `Pool<T>` interface.

The Art of Multiprocessor Programming. https://doi.org/10.1016/B978-0-12-415950-1.00020-3
Copyright © 2021 Elsevier Inc. All rights reserved.

  - A method is *partial* if calls may wait for certain conditions to hold. For example, a partial `get()` call that tries to remove an item from an empty pool blocks until an item is available to return. If the pool is bounded, a partial `put()` call that tries to add an item to a full pool blocks until an empty slot is available to fill. A partial interface makes sense when the producer (or consumer) has nothing better to do than to wait for the pool to become nonfull (or nonempty).
  - A method is *total* if calls never need to wait for conditions to become true. For example, a `get()` call that tries to remove an item from an empty pool, or a `put()` call that tries to add an item to a full pool, may immediately return a failure code or throw an exception. A total interface makes sense when the producer (or consumer) thread has something better to do than wait for the method call to take effect.
  - A partial method is *synchronous* if it waits for another method to overlap its call interval. For example, in a synchronous pool, a method call that adds an item to the pool is blocked until that item is removed by another method call. Symmetrically, a method call that removes an item from the pool is blocked until another method call makes an item available to be removed. Synchronous pools are used for communication in programming languages such as CSP and Ada, in which threads *rendezvous* to exchange information.
- Pools provide different *fairness* guarantees. They may be FIFO (i.e., a queue) or last-in-first-out (LIFO) (i.e., a stack), or have other, typically weaker, properties. The importance of fairness when buffering using a pool is clear to anyone who has ever called a bank or a technical support line, only to be placed in a pool of waiting calls. The longer you wait, the more consolation you draw from the recorded message asserting that calls are answered in the order they arrive. Perhaps.

10.2 Queues

In this chapter, we consider a kind of pool that provides *first-in-first-out* (FIFO) fairness. A sequential `Queue<T>` is an ordered sequence of items (of type `T`). It provides an `enq`(x) method that puts item x at one end of the queue, called the *tail*, and a `deq()` method that removes and returns the item at the other end of the queue, called the *head*. A concurrent queue is linearizable to a sequential queue. Queues are pools where `enq()` implements `put()` and `deq()` implements `get()`. We use queue implementations to illustrate a number of important principles. In later chapters we consider pools that provide other fairness guarantees.

10.3 A bounded partial queue

For simplicity, we assume it is illegal to add a *null* value to a queue. Of course, there may be circumstances where it makes sense to add and remove *null* values; we leave it as an exercise to adapt our algorithms to accommodate *null* values.

```
1  public class BoundedQueue<T> {
2    ReentrantLock enqLock, deqLock;
3    Condition notEmptyCondition, notFullCondition;
4    AtomicInteger size;
5    volatile Node head, tail;
6    final int capacity;
7    public BoundedQueue(int _capacity) {
8      capacity = _capacity;
9      head = new Node(null);
10     tail = head;
11     size = new AtomicInteger(0);
12     enqLock = new ReentrantLock();
13     notFullCondition = enqLock.newCondition();
14     deqLock = new ReentrantLock();
15     notEmptyCondition = deqLock.newCondition();
16   }
17   ...
18 }
```

FIGURE 10.2

The `BoundedQueue` class: fields and constructor.

```
19   protected class Node {
20     public T value;
21     public volatile Node next;
22     public Node(T x) {
23       value = x;
24       next = null;
25     }
26   }
```

FIGURE 10.3

The `BoundedQueue` class: list node.

How much concurrency can we expect a bounded queue implementation with multiple concurrent enqueuers and dequeuers to provide? Informally, the `enq()` and `deq()` methods operate on opposite ends of the queue; as long as the queue is neither full nor empty, an `enq()` call and a `deq()` call should be able to proceed without interference. For the same reason, concurrent `enq()` calls probably will interfere, and the same holds for `deq()` calls. This informal reasoning may sound convincing, and it is mostly correct, but realizing this level of concurrency is nontrivial.

Here, we implement a bounded queue as a linked list. (We could also have used an array.) Fig. 10.2 shows the queue's fields and constructor, and Fig. 10.3 shows a queue node. Figs. 10.4 and 10.5 show the `enq()` and `deq()` methods. Like the lists studied in Chapter 9, a queue node has `value` and `next` fields.

```
27  public void enq(T x) {
28    boolean mustWakeDequeuers = false;
29    Node e = new Node(x);
30    enqLock.lock();
31    try {
32      while (size.get() == capacity)
33        notFullCondition.await();
34      tail.next = e;
35      tail = e;
36      if (size.getAndIncrement() == 0)
37        mustWakeDequeuers = true;
38    } finally {
39      enqLock.unlock();
40    }
41    if (mustWakeDequeuers) {
42      deqLock.lock();
43      try {
44        notEmptyCondition.signalAll();
45      } finally {
46        deqLock.unlock();
47      }
48    }
49  }
```

FIGURE 10.4

The `BoundedQueue` class: the `enq()` method.

As shown in Fig. 10.6, the queue has `head` and `tail` fields that respectively refer to the first and last nodes in the list. The queue always contains at least one node, and the first node is a *sentinel*. Like the sentinel nodes in Chapter 9, it marks a position in the queue (in this case, the head of the queue), but its value is meaningless. Unlike the list algorithms in Chapter 9, in which the same nodes always act as sentinels, the queue repeatedly replaces the sentinel node. The abstraction map for this algorithm carries a list of nodes to a queue with the items referred to by the nonsentinel nodes in the list in the same order as they appear in the list. The item referred to by the first node is *not* in the abstract queue. The abstract queue is empty if there is only one node in the list (i.e., if `head.next == null`).

We use two locks, `enqLock` and `deqLock`, to ensure that at any time, at most one enqueuer and at most one dequeuer can manipulate the queue object's fields. Using two locks instead of one allows an enqueuer to not lock out a dequeuer unnecessarily, and vice versa. Each lock has an associated *condition*: `notFullCondition` for `enqLock` is used to notify waiting enqueuers when the queue is no longer full; `notEmptyCondition` for `deqLock` is used to notify waiting dequeuers when the queue is no longer empty.

To keep the queue bounded, we must prevent items from being enqueued when the queue is at capacity. The `size` field is an `AtomicInteger` that tracks the number

```
50  public T deq() {
51    T result;
52    boolean mustWakeEnqueuers = false;
53    deqLock.lock();
54    try {
55      while (head.next == null)
56        notEmptyCondition.await();
57      result = head.next.value;
58      head = head.next;
59      if (size.getAndDecrement() == capacity) {
60        mustWakeEnqueuers = true;
61      }
62    } finally {
63      deqLock.unlock();
64    }
65    if (mustWakeEnqueuers) {
66      enqLock.lock();
67      try {
68        notFullCondition.signalAll();
69      } finally {
70        enqLock.unlock();
71      }
72    }
73    return result;
74  }
```

FIGURE 10.5

The `BoundedQueue` class: the `deq()` method.

of objects currently in the queue. This field is decremented by `deq()` calls and incremented by `enq()` calls. We use an `AtomicInteger` because this field is not protected by either lock: An enqueuer and a dequeuer may access it concurrently.

To enqueue an item, a thread acquires the `enqLock` (line 30), and reads the `size` field (line 32). If that field is equal to the capacity, the queue is full, and the enqueuer must wait until a dequeuer makes room. The enqueuer waits on `notFullCondition` (line 33), releasing the `enqLock` temporarily, and blocking until that condition is signaled. Each time the thread awakens, it checks whether there is room; if not, it goes back to sleep.

Once the enqueuer determines there is room, it can proceed to completion. No other thread can fill the queue while the enqueue is in progress: All other enqueuers are locked out, and concurrent dequeuers only increase the space available.

We must carefully check that this implementation does not suffer from the kind of "lost-wakeup" bug described in Chapter 8. Care is needed because an enqueuer encounters a full queue in two steps: First, it sees that `size` is the queue capacity, and second, it waits on `notFullCondition` until there is room in the queue. When

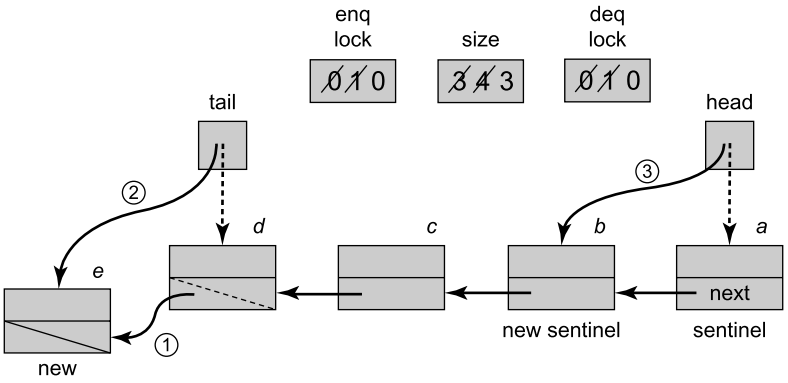


FIGURE 10.6

The `enq()` and `deq()` methods of the `BoundedQueue` with four slots. First a node is enqueued into the queue by acquiring the `enqLock`. The `enq()` checks that the `size` is 3, which is less than the bound. It then redirects the `next` field of the node referenced by the `tail` field (step 1), redirects `tail` to the new node (step 2), increments the `size` to 4, and releases the lock. Since `size` is now 4, any further calls to `enq()` will cause the threads to block until the `notFullCondition` is signaled by some `deq()`. Next, a node is dequeued from the queue by some thread. The `deq()` acquires the `deqLock`, reads the new value *b* from the successor of the node referenced by `head` (this node is the current sentinel), redirects `head` to this successor node (step 3), decrements the `size` to 3, and releases the lock. Before completing the `deq()`, because the `size` was 4 when it started, the thread acquires the `enqLock` and signals any enqueuers waiting on `notFullCondition` that they can proceed.

a dequeuer changes the queue from full to nonfull, it acquires `enqLock` and signals `notFullCondition`. Even though the `size` field is not protected by the `enqLock`, the dequeuer acquires the `enqLock` before it signals the condition, so the dequeuer cannot signal between the enqueuer's two steps.

To dequeue an item, a thread acquires the `deqLock` and checks whether the queue is empty. However, unlike in the `enq()` method, a dequeuer does not read the `size` field. Instead, it checks whether `head.next == null` (line 55); if so, the abstract queue is empty and the thread must wait until an item is enqueued. Like in the `enq()` method, the dequeuer waits on `notEmptyCondition`, which temporarily releases `deqLock`, and blocks until the condition is signaled. Each time the thread awakens, it checks whether the queue is empty, and if so, goes back to sleep.

Once a dequeuer establishes that the queue is nonempty, the queue will remain nonempty for the duration of the `deq()` call, because all other dequeuers have been locked out. Because the queue is nonempty, it has a nonsentinel node; the dequeuer accesses the first such node (i.e., the node referenced by the sentinel node's `next` field). It reads this node's `value` field, and makes the node the new sentinel node by setting the queue's `head` to refer to it. The dequeuer then decrements `size` and releases the `deqLock`. If the dequeuer found the former size was the queue capacity, then there may be enqueuers waiting on `notEmptyCondition`, so the dequeuer acquires `enqLock`, and signals all such threads to wake up.

Note that the abstract queue's last item is not always the one in the node referenced by `tail`. An item is logically added to the queue as soon as the last node's `next` field is redirected to the new node, even if the enqueuer has not yet updated `tail` (i.e., an `enq()` call linearizes to line 34). For example, suppose a thread is in the process of inserting a new node: It has acquired the `enqLock` and redirected the last node to point to the new node, but has not yet redirected the `tail` field. A concurrent dequeuing thread could acquire the `deqLock`, read and return the new node's value, redirect the `head` to the new node, and decrement `size`, all before the enqueuer redirects `tail` to the newly inserted node. In this example, `size` would be negative temporarily because the dequeuer decrements it before the enqueuer increments it. The enqueuer need not wake any waiting dequeuers in this case, because the item it enqueued has already been dequeued.

One drawback of this implementation is that concurrent `enq()` and `deq()` calls interfere with each other, but not through locks. All method calls apply `getAndIncrement()` or `getAndDecrement()` calls to the `size` field. These methods are more expensive than ordinary reads and writes, and they could cause a sequential bottleneck.

We can reduce such interactions by splitting this field into two: `enqSideSize` is an integer field incremented by `enq()`, and `deqSideSize` is an integer field decremented by `deq()`; the actual size of the queue is the sum of these two counters (`deqSideSize` is always 0 or negative). A thread calling `enq()` tests `enqSideSize`, and as long as it is less than the capacity, it proceeds. When the field reaches capacity, the thread locks `deqLock`, adds `deqSideSize` to `enqSideSize`, and resets `deqSideSize` to 0. Instead of synchronizing on every method call, this technique synchronizes sporadically when the enqueuer's size estimate becomes too large.

10.4 An unbounded total queue

We now describe an implementation of an unbounded queue. The `enq()` method always enqueues its item, and `deq()` throws `EmptyException` if there is no item to dequeue. The representation is the same as the bounded queue, except there is no need to count the number of items in the queue, or to provide conditions on which to wait. As shown in Figs. 10.7 and 10.8, this algorithm is simpler than the bounded algorithm.

This queue cannot deadlock, because each method acquires only one lock, either `enqLock` or `deqLock`. A sentinel node alone in the queue will never be deleted, so each `enq()` call will succeed as soon as it acquires the lock. Of course, a `deq()` method may fail if the queue is empty (i.e., if `head.next` is *null*). As in the bounded queue implementation, an item is actually enqueued when the `enq()` call sets the last node's `next` field to the new node, even before `enq()` resets `tail` to refer to the new node. After that instant, the new item is reachable along a chain of the `next` references. As usual, the queue's actual head and tail are not necessarily the items referenced by `head` and `tail`. Instead, the actual head is the item reference by the successor of `head`,

```
1   public void enq(T x) {
2     Node e = new Node(x);
3     enqLock.lock();
4     try {
5       tail.next = e;
6       tail = e;
7     } finally {
8       enqLock.unlock();
9     }
10  }
```

FIGURE 10.7

The UnboundedQueue<T> class: the enq() method.

```
11  public T deq() throws EmptyException {
12    T result;
13    deqLock.lock();
14    try {
15      if (head.next == null) {
16        throw new EmptyException();
17      }
18      result = head.next.value;
19      head = head.next;
20    } finally {
21      deqLock.unlock();
22    }
23    return result;
24  }
```

FIGURE 10.8

The UnboundedQueue<T> class: the deq() method.

and the actual tail is the last item reachable from the head. Both the enq() and deq() methods are total as they do not wait for the queue to become empty or full.

10.5 A lock-free unbounded queue

We now describe a lock-free unbounded queue implementation. Figs. 10.9–10.12 show the LockFreeQueue<T> class, a natural extension of the unbounded total queue of Section 10.4. It prevents method calls from starving by having the quicker threads help the slower threads.

As before, we represent the queue as a list of nodes, in which the first node is a sentinel whose value is meaningless. However, as shown in Figs. 10.9 and 10.10,

```
1  public class LockFreeQueue<T> {
2    AtomicReference<Node> head, tail;
3    public LockFreeQueue() {
4      Node node = new Node(null);
5      head = new AtomicReference(node);
6      tail = new AtomicReference(node);
7    }
8    ...
9  }
```

FIGURE 10.9

The `LockFreeQueue<>` class: fields and constructor.

```
10   public class Node {
11     public T value;
12     public AtomicReference<Node> next;
13     public Node(T value) {
14       this.value = value;
15       next = new AtomicReference<Node>(null);
16     }
17   }
```

FIGURE 10.10

The `LockFreeQueue<T>` class: list node.

```
18   public void enq(T value) {
19     Node node = new Node(value);
20     while (true) {
21       Node last = tail.get();
22       Node next = last.next.get();
23       if (last == tail.get()) {
24         if (next == null) {
25           if (last.next.compareAndSet(next, node)) {
26             tail.compareAndSet(last, node);
27             return;
28           }
29         } else {
30           tail.compareAndSet(last, next);
31         }
32       }
33     }
34   }
```

FIGURE 10.11

The `LockFreeQueue<T>` class: the `enq()` method.

```
35  public T deq() throws EmptyException {
36    while (true) {
37      Node first = head.get();
38      Node last = tail.get();
39      Node next = first.next.get();
40      if (first == head.get()) {
41        if (first == last) {
42          if (next == null) {
43            throw new EmptyException();
44          }
45          tail.compareAndSet(last, next);
46        } else {
47          T value = next.value;
48          if (head.compareAndSet(first, next))
49            return value;
50        }
51      }
52    }
53  }
```

FIGURE 10.12

The `LockFreeQueue<T>` class: the `deq()` method.

`head` and `tail` fields are `AtomicReference<Node>` fields that refer to the first node and the last node in the queue, respectively, and each node's `next` field is an `AtomicReference<Node>` that refers to the next node in the list. The queue constructor creates a new sentinel node and sets both `head` and `tail` to refer to it.

The `enq()` method (Fig. 10.11) creates a new node (line 19), locates the last node in the queue (lines 21–22), and then updates the list to append the new node. This method is *lazy*: It does the update in two distinct steps, illustrated in Fig. 10.13:

1. it calls `compareAndSet()` to append the new node (line 25), and then
2. it calls `compareAndSet()` to change the queue's `tail` field from the prior last node to the new last node (line 26).

Because these two steps are not executed atomically, every other method call must be prepared to encounter a half-finished `enq()` call, and to finish the job. This is a real-world example of the "helping" technique we first saw in the universal construction of Chapter 6.

We now review all the steps in detail. An enqueuer thread *A* creates a new node with the new value to be enqueued (line 19), and finds the node that appears to be last by reading `tail` (line 21–23). To verify that the node found is indeed last, *A* checks that it has no successor (line 24). If so, *A* attempts to append the new node by calling `compareAndSet()` (line 25). (A `compareAndSet()` is required because other threads may be trying the same thing.) If the `compareAndSet()` succeeds, *A* uses a sec-

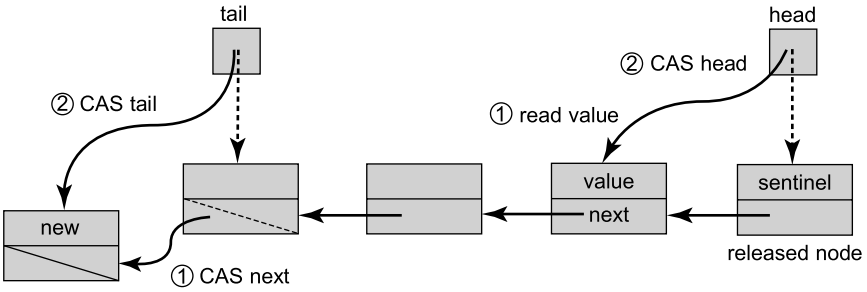


FIGURE 10.13

The `enq()` and `deq()` methods of the `LockFreeQueue`. The `enq()` method is lazy: a node is inserted into the queue in two steps. First, a `compareAndSet()` call changes the `next` field of the node referenced by the queue's `tail` from *null* to the new node. Then a `compareAndSet()` call advances `tail` itself to refer to the new node. An item is removed from the queue by checking that the sentinel has a successor, and then calling `compareAndSet()` to redirect `head` from the current sentinel to its successor, making the latter the new sentinel. The item removed is the one referred to by the new sentinel. Both `enq()` and `deq()` methods help complete unfinished `tail` updates.

ond `compareAndSet()` to advance `tail` to the new node (line 26). Even if this second `compareAndSet()` call fails, *A* can still return successfully because, as we will see, this `compareAndSet()` fails only if some other thread "helped" *A* by advancing `tail`.

If the tail node has a successor (line 29), then some other enqueuer must have appended its node but not updated `tail` before *A* read it. In this case, *A* tries to "help" that other thread by advancing `tail` to refer directly to the successor (line 30) before trying again to insert its own node.

This `enq()` is total, meaning that it never waits for a dequeuer. A successful `enq()` is linearized at the instant where the executing thread (or a concurrent helping thread) successfully calls `compareAndSet()` to redirect the `tail` field to the new node at line 30.

The `deq()` method is similar to its counterpart from the `UnboundedQueue`. If the queue is nonempty, the dequeuer calls `compareAndSet()` to change `head` from the sentinel node to its successor, making the successor the new sentinel node. The `deq()` method makes sure that the queue is not empty in the same way as before: by checking that the `next` field of the `head` node is not *null*.

There is, however, a subtle issue in the lock-free case, depicted in Fig. 10.14: Before advancing `head`, a dequeuer must make sure that `tail` is not left referring to the sentinel node that is about to be removed from the queue. To avoid this problem we add a test: If `head` equals `tail` (line 41) and the (sentinel) node they refer to has a *nonnull* `next` field (line 42), then the `tail` is deemed to be lagging behind. In this case, as in the `enq()` method, the dequeuer attempts to help make `tail` consistent by swinging it to the sentinel node's successor (line 45), and only then updates `head` to remove the sentinel (line 48). As in the partial queue, the value is read from the successor of the sentinel node (line 47). If this method returns a value, then its lin-

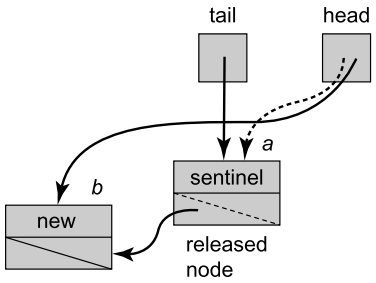


FIGURE 10.14

Why dequeuers must help advance `tail` in line 45 of Fig. 10.12. Consider the scenario in which a thread enqueuing node *b* has redirected *a*'s `next` field to *b*, but has yet to redirect `tail` from *a* to *b*. If another thread starts dequeuing, it will read *b*'s value and redirect `head` from *a* to *b*, effectively removing node *a* while `tail` still refers to it. To avoid this problem, the dequeuing thread must help advance `tail` from *a* to *b* before redirecting `head`.

earization point occurs when it successfully appends a node to the list (i.e., when the `compareAndSet()` at line 48 succeeds); otherwise it is linearized when it saw that the sentinel node has no successor (i.e., when it got a *null* value at line 39).

It is easy to check that the resulting queue is lock-free. Every method call first checks for an incomplete `enq()` call, and tries to complete it. In the worst case, all threads are trying to advance the queue's `tail` field, and one of them must succeed. A thread fails to enqueue or dequeue a node only if another thread's method call succeeds in changing the reference, so some method call always completes. As it turns out, being lock-free substantially enhances the performance of queue implementations, and lock-free algorithms often outperform the most efficient blocking ones.

10.6 Memory reclamation and the ABA problem

Our queue implementations so far rely on the Java garbage collector to recycle nodes after they have been dequeued. What happens if we choose to do our own memory management? There are several reasons why we might want to do this. Languages such as C or C++ do not provide garbage collection. Even if garbage collection is available, it is often more efficient for a class to do its own memory management, particularly if it creates and releases many small objects. Finally, if the garbage collection process is not lock-free, we might want to supply our own lock-free memory reclamation.

A natural way to recycle nodes in a lock-free manner is to have each thread maintain its own private (i.e., thread-local) *free list* of unused queue entries.

```
ThreadLocal<Node> freeList = new ThreadLocal<Node>() {
  protected Node initialValue() { return null; };
};
```

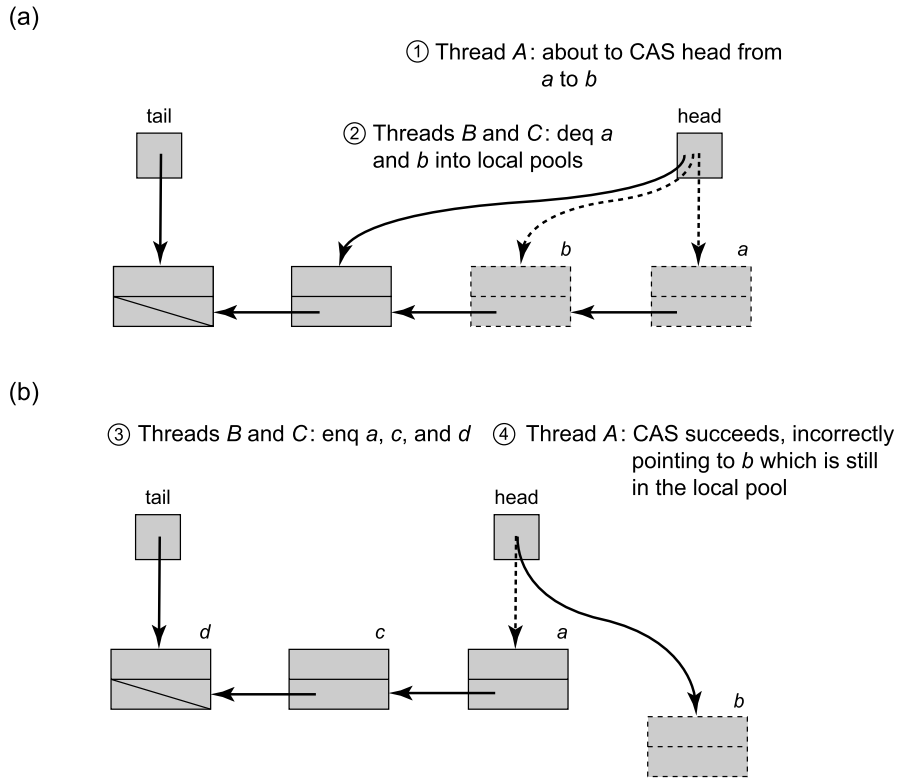


FIGURE 10.15

An ABA scenario: Assume that we use local pools of recycled nodes in our lock-free queue algorithm. In part (a), the dequeuer thread *A* observes that the sentinel node is *a*, and next node is *b*. (Step 1) It then prepares to update `head` by applying a `compareAndSet()` with old value *a* and new value *b*. (Step 2) Suppose, however, that before it takes another step, other threads dequeue *b*, then its successor, placing both *a* and *b* in the free pool. In part (b), (Step 3) node *a* is reused, and eventually reappears as the sentinel node in the queue. (Step 4) Thread *A* now wakes up, calls `compareAndSet()`, and succeeds in setting `head` to *b*, since the old value of `head` is indeed *a*. Now, `head` is incorrectly set to a recycled node.

When an enqueuing thread needs a new node, it tries to remove one from its thread-local free list. If the free list is empty, it simply allocates a node using the **`new`** operator. When a dequeuing thread is ready to retire a node, it links it back onto the thread-local list. Because the list is thread-local, there is no need for expensive synchronization. This design works well as long as each thread performs roughly the same number of enqueues and dequeues. If there is an imbalance, then there may be a need for more complex techniques, such as periodically stealing nodes from other threads.

Surprisingly, perhaps, the lock-free queue will not work if nodes are recycled in the most straightforward way. Consider the scenario depicted in Fig. 10.15. In

part (a), the dequeuing thread *A* observes the sentinel node is *a*, and the next node is *b*. It then prepares to update `head` by calling `compareAndSet()` with old value *a* and new value *b*. Before it takes another step, other threads dequeue *b* and its successor, placing both *a* and *b* in the free pool. Node *a* is recycled, and eventually reappears as the sentinel node in the queue, as depicted in part (b). The thread now wakes up, calls `compareAndSet()`, and succeeds, since the old value of the `head` is indeed *a*. Unfortunately, it has redirected `head` to a recycled node!

This phenomenon is called the *ABA problem*. It shows up often, especially in dynamic memory algorithms that use conditional synchronization operations such as `compareAndSet()`. Typically, a reference about to be modified by a `compareAndSet()` changes from *a* to *b* and back to *a* again. As a result, the `compareAndSet()` call succeeds even though its effect on the data structure has changed, and no longer has the desired effect.

One straightforward way to fix this problem is to tag each atomic reference with a unique *stamp*. An `AtomicStampedReference<T>` object, described in detail in Pragma 10.6.1, encapsulates both a reference to an object of Type `T` and an integer *stamp*. These fields can be atomically updated either together or individually.

PRAGMA 10.6.1

The `AtomicStampedReference<T>` class encapsulates both a reference to an object of Type `T` and an integer *stamp*. It generalizes the `AtomicMarkableReference<T>` class (Pragma 9.8.1), replacing the Boolean *mark* with an integer stamp.

We most commonly use this stamp as a *version number* to avoid the ABA problem, incrementing the value of the stamp each time we modify the object. Sometimes, as in the `LockFreeExchanger<>` class of Chapter 11, we use the stamp to hold one of a finite set of states.

The stamp and reference fields can be updated atomically, either together or individually. For example, the `compareAndSet()` method tests expected reference and stamp values, and if both tests succeed, replaces them with updated reference and stamp values. The `get()` method has an unusual interface: It returns the object's reference value and stores the stamp value in an integer array argument. Fig. 10.16 illustrates the signatures for these methods.

In a language like C or C++, one could provide this functionality efficiently in a 64-bit architecture by "stealing" bits from pointers. A 32-bit architecture would probably require a level of indirection.

Fig. 10.17 shows the `deq()` method using the `AtomicStampedReference<Node>` to avoid the ABA problem. Each time through the loop, it reads both the reference and stamp values for the first, next, and last nodes (lines 6–8). It uses `compareAndSet()` to compare both the reference and the stamp (line 18). It increments the stamp each time it uses `compareAndSet()` to update a reference (lines 15 and 18).[1]

[1] For simplicity, we ignore the (remote) possibility that the stamp could wrap around and cause an error.

```
1  public boolean compareAndSet(T expectedReference,
2                               T newReference,
3                               int expectedStamp,
4                               int newStamp);
5  public T get(int[] stampHolder);
6  public T getReference();
7  public int getStamp();
8  public void set(T newReference, int newStamp);
```

FIGURE 10.16

The `AtomicStampedReference<T>` class: the `compareAndSet()` and `get()` methods. The `compareAndSet()` method tests and updates both the stamp and reference fields; the `get()` method returns the encapsulated reference and stores the stamp at position 0 in the argument array; the `getReference()` and `getStamp()` methods return the reference and stamp, respectively; and the `put()` method updates the encapsulated reference and the stamp.

```
1   public T deq() throws EmptyException {
2     int[] lastStamp = new int[1];
3     int[] firstStamp = new int[1];
4     int[] nextStamp = new int[1];
5     while (true) {
6       Node first = head.get(firstStamp);
7       Node last = tail.get(lastStamp);
8       Node next = first.next.get(nextStamp);
9       if (head.getStamp() == firstStamp[0]) {
10        if (first == last) {
11          if (next == null) {
12            throw new EmptyException();
13          }
14          tail.compareAndSet(last, next,
15              lastStamp[0], lastStamp[0]+1);
16        } else {
17          T value = next.value;
18          if (head.compareAndSet(first, next, firstStamp[0],
                        firstStamp[0]+1)) {
19            free(first);
20            return value;
21          }
22        }
23      }
24    }
25  }
```

FIGURE 10.17

The `LockFreeQueueRecycle<T>` class: The `deq()` method uses stamps to avoid ABA.

The ABA problem can occur in many synchronization scenarios, not just those involving conditional synchronization. For example, it can occur when using only loads and stores. Conditional synchronization operations such as *load-linked/store-conditional*, available on some architectures (see Appendix B), avoid the ABA problem by testing not whether a value is the same at two points in time, but whether the value has ever changed between those points.

10.6.1 A naïve synchronous queue

We now turn our attention to an even tighter kind of synchronization. One or more *producer* threads produce items to be removed, in FIFO order, by one or more *consumer* threads. Here, however, producers and consumers *rendezvous* with one another: A producer that puts an item in the queue blocks until that item is removed by a consumer, and vice versa. Such rendezvous synchronization is built into languages such as CSP and Ada.

Fig. 10.18 shows he `SynchronousQueue<T>` class, a straightforward monitor-based synchronous queue implementation. It has the following fields: `item` is the first item waiting to be dequeued, `enqueuing` is a Boolean value used by enqueuers to synchronize among themselves, `lock` is the lock used for mutual exclusion, and `condition` is used to block partial methods. If the `enq()` method finds `enqueuing` to be *true* (line 10), then another enqueuer has supplied an item and is waiting to rendezvous with a dequeuer, so the enqueuer repeatedly releases the lock, sleeps, and, when it awakens, checks whether `enqueuing` has become *false* (line 11). When this condition is satisfied, the enqueuer sets `enqueuing` to *true*, which locks out other enqueuers until the current rendezvous is complete, and sets `item` to refer to the new item (lines 12–13). It then notifies any waiting threads (line 14), and waits until `item` becomes *null* (lines 15–16). When the wait is over, the rendezvous has occurred, so the enqueuer sets `enqueuing` to *false*, notifies any waiting threads, and returns (lines 17 and 19).

The `deq()` method simply waits until `item` is not *null* (lines 26–27), records the item, sets the `item` field to *null*, and notifies any waiting threads before returning the item (lines 28–31).

Although the design of the queue is relatively simple, it incurs a high synchronization cost. Whenever one thread might wake up another, both enqueuers and dequeuers wake up all waiting threads, leading to a number of wakeups quadratic in the number of waiting threads. Although it is possible to use multiple condition objects to reduce the number of wakeups, it is still necessary to block on every call, which is expensive.

10.7 Dual data structures

To reduce the synchronization overheads of the synchronous queue, we consider an alternative synchronous queue implementation that treats `enq()` and `deq()` methods symmetrically, splitting a `deq()` method call that finds the queue empty into two steps. In the first step, the dequeuer puts a *reservation* object in the queue, indicating that

```
1  public class SynchronousQueue<T> {
2    T item = null;
3    boolean enqueuing;
4    Lock lock;
5    Condition condition;
6    ...
7    public void enq(T value) {
8      lock.lock();
9      try {
10       while (enqueuing)
11         condition.await();
12       enqueuing = true;
13       item = value;
14       condition.signalAll();
15       while (item != null)
16         condition.await();
17       enqueuing = false;
18       condition.signalAll();
19     } finally {
20       lock.unlock();
21     }
22   }
23   public T deq() {
24     lock.lock();
25     try {
26       while (item == null)
27         condition.await();
28       T t = item;
29       item = null;
30       condition.signalAll();
31       return t;
32     } finally {
33       lock.unlock();
34     }
35   }
36 }
```

FIGURE 10.18

The `SynchronousQueue<T>` class.

the dequeuer is waiting for an enqueuer with which to rendezvous. The reservation object contains an empty slot, on which the dequeuer spins until the slot is occupied; an enqueuer *fulfills* the reservation by depositing an item into that slot. Similarly, when an enqueuer adds an item to the queue, if there is no reservation to fulfill, it spins on the item until it is removed by a dequeuer. The queue contains either only

```
1  private enum NodeType {ITEM, RESERVATION};
2  private class Node {
3    volatile NodeType type;
4    volatile AtomicReference<T> item;
5    volatile AtomicReference<Node> next;
6    Node(T myItem, NodeType myType) {
7      item = new AtomicReference<T>(myItem);
8      next = new AtomicReference<Node>(null);
9      type = myType;
10   }
11 }
```

FIGURE 10.19

The `SynchronousDualQueue<T>` class: queue node.

items waiting to be dequeued or only reservations waiting to be fulfilled, or it is empty; it never contains items and reservations at the same time.

This structure is called a *dual data structure*, because it can contain both items and reservations. It has a number of nice properties. First, waiting threads can spin on a locally cached flag, which we have seen is essential for scalability. Second, it ensures fairness in a natural way. Reservations are queued in the order they arrive, ensuring that requests are fulfilled in the same order. Note that this data structure is linearizable, since each partial method call can be ordered when it is fulfilled.

The queue is implemented as a list of nodes, where a node represents either an item waiting to be dequeued, or a reservation waiting to be fulfilled (Fig. 10.19). A node's `type` field indicates which. At any time, all nodes in the queue have the same type: Either the queue consists entirely of items waiting to be dequeued, or entirely of reservations waiting to be fulfilled.

When an item is enqueued, the node's `item` field holds the item, which is reset to *null* when that item is dequeued. When a reservation is enqueued, the node's `item` field is *null*, and is reset to an item when fulfilled by an enqueuer.

Fig. 10.20 shows the `SynchronousDualQueue`'s constructor and `enq()` method. (The `deq()` method is symmetric.) As in earlier queues we have considered, the `head` field always refers to a *sentinel* node that serves as a placeholder, and whose actual value (and type) is unimportant. The queue is empty when `head` and `tail` refer to the same node (i.e., the sentinel node). The constructor creates a sentinel node with an arbitrary value, referred to by both `head` and `tail`.

The `enq()` method first checks whether the queue is empty or contains enqueued items waiting to be dequeued (line 21). If so, then just as in the lock-free queue, it reads the queue's `tail` field (line 22), and checks that the values read are consistent (line 23). If the `tail` field does not refer to the last node in the queue, then the method advances the `tail` field and starts over (lines 24–25). Otherwise, the `enq()` method tries to append the new node to the end of the queue by resetting the tail node's `next` field to refer to the new node (line 26). If it succeeds, it tries to advance the tail to the

```
12  public SynchronousDualQueue() {
13    Node sentinel = new Node(null, NodeType.ITEM);
14    head = new AtomicReference<Node>(sentinel);
15    tail = new AtomicReference<Node>(sentinel);
16  }
17  public void enq(T e) {
18    Node offer = new Node(e, NodeType.ITEM);
19    while (true) {
20      Node t = tail.get(), h = head.get();
21      if (h == t || t.type == NodeType.ITEM) {
22        Node n = t.next.get();
23        if (t == tail.get()) {
24          if (n != null) {
25            tail.compareAndSet(t, n);
26          } else if (t.next.compareAndSet(n, offer)) {
27            tail.compareAndSet(t, offer);
28            while (offer.item.get() == e);
29            h = head.get();
30            if (offer == h.next.get())
31              head.compareAndSet(h, offer);
32            return;
33          }
34        }
35      } else {
36        Node n = h.next.get();
37        if (t != tail.get() || h != head.get() || n == null) {
38          continue;
39        }
40        boolean success = n.item.compareAndSet(null, e);
41        head.compareAndSet(h, n);
42        if (success)
43          return;
44      }
45    }
46  }
```

FIGURE 10.20

The `SynchronousDualQueue<T>` class: `enq()` method and constructor.

newly appended node (line 27), and then spins, waiting for a dequeuer to announce that it has dequeued the item by setting the node's `item` field to *null*. Once the item is dequeued, the method tries to clean up by making its node the new sentinel. This last step serves only to enhance performance, because the implementation remains correct, whether or not the method advances the `head` reference.

If, however, the `enq()` method discovers that the queue contains dequeuers' reservations waiting to be fulfilled, then it tries to find a reservation to fulfill. Since the

queue's `head` node is a sentinel with no meaningful value, `enq()` reads the head's successor (line 36), checks that the values it has read are consistent (lines 37–39), and tries to switch that node's `item` field from *null* to the item being enqueued. Whether or not this step succeeds, the method tries to advance `head` (line 41). If the `compareAndSet()` call succeeds (line 40), the method returns; otherwise it retries.

10.8 Chapter notes

The partial queue employs a mixture of techniques adapted from Doug Lea [110] and from an algorithm by Maged Michael and Michael Scott [125]. The lock-free queue is a slightly simplified version of a queue algorithm by Maged Michael and Michael Scott [125]. The synchronous queue implementations are adapted from algorithms by Bill Scherer, Doug Lea, and Michael Scott [167].

10.9 Exercises

Exercise 10.1. Change the `SynchronousDualQueue<T>` class to work correctly with *null* items.

Exercise 10.2. Consider the queue presented in Fig. 10.21, a variant of the simple lock-free queue for a single enqueuer and a single dequeuer described in Chapter 3. This queue is blocking; that is, removing an item from an empty queue, or adding an item to a full one, causes the threads to spin. Surprisingly, this queue requires only loads and stores and not a more powerful read–modify–write synchronization operation.

Does the queue implementation, however, require the use of a memory barrier? If so, where in the code is such a barrier needed and why? If not, explain why not.

Exercise 10.3. Design a bounded lock-based queue implementation using an array instead of a linked list.

1. Allow parallelism by using two separate locks for `head` and `tail`.
2. Try to transform your algorithm to be lock-free. Where do you run into difficulty?

Exercise 10.4. In the `deq()` method of the unbounded lock-based queue (Fig. 10.8), is it necessary to hold the lock when checking that the queue is not empty? Explain.

Exercise 10.5. In Dante's *Inferno*, he describes a visit to Hell. In a recently discovered chapter, he encounters five people sitting at a table with a pot of stew in the middle. Although each one holds a spoon that reaches the pot, each spoon's handle is much longer than each person's arm, so no one can feed him- or herself. They are famished and desperate.

Dante then suggests: "Why do you not feed one another?"

The rest of the chapter is lost.

```
1  class TwoThreadLockFreeQueue<T> {
2    int head = 0, tail = 0;
3    T[] items;
4    public TwoThreadLockFreeQueue(int capacity) {
5      head = 0; tail = 0;
6      items = (T[]) new Object[capacity];
7    }
8    public void enq(T x) {
9      while (tail - head == items.length) {};
10     items[tail % items.length] = x;
11     tail++;
12   }
13   public Object deq() {
14     while (tail - head == 0) {};
15     Object x = items[head % items.length];
16     head++;
17     return x;
18   }
19 }
```

FIGURE 10.21

A lock-free FIFO queue with blocking semantics for a single enqueuer and single dequeuer. The queue is implemented in an array. Initially the `head` and `tail` fields are equal and the queue is empty. If the `head` and `tail` differ by `capacity`, then the queue is full. The `enq()` method reads the `head` field, and if the queue is full, it repeatedly checks the `head` until the queue is no longer full. It then stores the object in the array, and increments the `tail` field. The `deq()` method works in a symmetric way.

1. Write an algorithm to allow these unfortunates to feed one another. Two or more people may not feed the same person at the same time. Your algorithm must be, well, starvation-free.
2. Discuss the advantages and disadvantages of your algorithm. Is it centralized or decentralized, high or low in contention, and deterministic or randomized?

Exercise 10.6. Consider the linearization points of the `enq()` and `deq()` methods of the lock-free queue (Figs.10.11 and 10.12).

1. Can we choose the point at which the returned value is read from a node as the linearization point of a successful `deq()`? Explain.
2. Can we choose the linearization point of the `enq()` method to be the point at which the `tail` field is updated, possibly by other threads? Explain.

Exercise 10.7. Consider the unbounded queue implementation shown in Fig. 10.22. This queue is blocking, meaning that the `deq()` method does not return until it has found an item to dequeue.

The queue has two fields: `items` is a very large array, and `tail` is the index of the next unused element in the array.

```
1  public class HWQueue<T> {
2    AtomicReference<T>[] items;
3    AtomicInteger tail;
4    ...
5    public void enq(T x) {
6      int i = tail.getAndIncrement();
7      items[i].set(x);
8    }
9    public T deq() {
10     while (true) {
11      int range = tail.get();
12      for (int i = 0; i < range; i++) {
13         T value = items[i].getAndSet(null);
14         if (value != null) {
15           return value;
16         }
17       }
18     }
19   }
20 }
```

FIGURE 10.22

Queue used in Exercise 10.7.

1. Are the `enq()` and `deq()` methods wait-free? If not, are they lock-free? Explain.
2. Identify linearization points for the `enq()` and `deq()` methods. (Careful! They may be execution-dependent.)